

Stoquify Transaction History, Movement Analytics, Security Hashing, and HRIS Proposal

Date: 2026-07-14

Workspace: E:\ohada saas\Focused projects\stoquify

Prompt saved as: docs/new ideas/STOQUIFY_TRANSACTION_HISTORY_ANALYTICS_AND_SECURITY_EXECUTION_PROMPT_2026-07-14.md

Executive Summary

Stoquify already has meaningful transaction-history foundations. The strongest current areas are inventory movements, cash drawer sessions and journal entries, AP controls, payment reconciliation, payroll proof workbenches, accounting journals, and close assurance evidence. The weaker areas are not absence of data, but incomplete operational presentation: several surfaces are recent-activity snippets, capped arrays, client-filtered exports, or aggregate dashboards rather than complete, paginated, service-owned ledgers.

The best direction is not to copy `/en/dashboard/inventory/movements` into every domain. The better product move is a shared, server-backed `TransactionHistoryWorkbench` pattern with domain-owned read models. It should preserve Stoquify's authenticated dashboard command-center style while ensuring that business truth stays in services, not UI tables.

The proposal is to build a "trustable operations history" layer:

- A shared UI anatomy: command brief, KPI/status strip, action queue, filter bar, workbench table, and proof/detail drawer.
- Domain-owned source-of-truth services for supplier payables, customer receivables, cashier settlement, payment reconciliation, inventory adjustments, AP controls, sales/AR controls, payroll proof, and close evidence.
- A shared row contract that shows state, risk, action, and proof without flattening every domain into a lossy universal ledger.
- A practical security hardening track that clarifies when SHA-256 is appropriate and where Argon2id, HMAC-SHA-256, public-key signatures, key rotation, expiry, and redaction are needed.
- A phased HRIS roadmap that recognizes the existing payroll-adjacent HRIS foundation without pretending Stoquify already has a standalone HRIS module.

Method And Specialist Inputs

This proposal used repo inspection plus six focused specialist reviews:

- Codebase orientation: routes, services, Prisma models, UI surfaces, and gaps.
- UX architecture: command-center dashboard patterns and history workbench design.
- Security architecture: hashing, signing, token, audit, and privacy implications.
- Bookkeeping/control: accounting value, close impact, reconciliation, subledger risks.
- HRIS review: current payroll-adjacent HRIS foundation and roadmap.
- Product strategy: prioritization and product moat.

The assessment is static and implementation-oriented. It does not claim production data quality, statutory expert certification, or runtime tie-out results.

Confirmed Current System Behavior

Inventory Movements

Confirmed:

- `/en/dashboard/inventory/movements` renders `StockMovementDashboard`.
- The route requires `inventory.levels.read`.
- The page supports item, location, type, and date filters.
- The service returns inventory transactions in descending organization-scoped order, with default limit 100 and service cap 500.
- Prisma has `InventoryTransaction` with item, location, organization, user, reference, batch/serial/expiry, signed quantity, cost fields, and `balanceAfter`.

Important gaps:

- The visible table requests a fixed 100 rows and lacks durable cursor pagination.
- Summary metrics are not fully aligned to every visible table filter.
- `WRITE_OFF` exists in the Prisma enum but is not consistently represented in the current movement UI configuration.
- Some reserved quantity fields are placeholder zero values in the read model.
- Inventory movement is a good visual reference, but not yet a complete platform-wide history contract.

Relevant evidence:

- `app/[locale]/(dashboard)/dashboard/inventory/movements/page.tsx`
- `components/inventory/movements/StockMovementDashboard.tsx`
- `actions/inventory/inventoryMovementActions.ts`
- `services/inventory/inventory-read.service.ts`
- `prisma/schema.prisma`

Supplier And Customer Detail Activity

Confirmed:

- Supplier and customer management dashboards include analytics/detail views.
- Supplier detail analytics retrieve recent purchase orders and ledger entries.
- Customer detail analytics retrieve recent sales orders, ledger entries, and payments.
- Prisma includes `SupplierLedgerEntry` and `CustomerLedgerEntry`.
- Supplier models include invoices, payments, payment allocations, bank-account change controls, current balance, and ledger entries.
- Customer models include current balance, sales orders, and ledger entries.

Important gaps:

- The existing supplier and customer detail histories are recent snippets, not full statements.
- They are useful for quick inspection, but not sufficient as complete payable/receivable ledgers.
- A full supplier payable or customer receivable statement must be service-owned and tie to invoices, allocations, ledger entries, source links, reconciliation state, and accounting postings.

Relevant evidence:

- `services/supplier/supplier.service.ts`
- `services/customer/customer.service.ts`
- `components/suppliers/SupplierManagementDashboard.tsx`
- `components/customers/CustomerManagementDashboard.tsx`
- `prisma/schema.prisma`

Cashier And POS Movement

Confirmed:

- Cash drawer reporting already includes session history and a movement-style journal.
- Prisma has `POSSession`, `CashDrawer`, and `CashDrawerTransaction`.
- The drawer dashboard service composes sessions, cash movements, tender totals, expected balances, counted balances, variances, alerts, and journal rows.
- The UI shows drawer sessions and recent journal events.

Important gaps:

- The dashboard is bounded and not a complete cashier transaction ledger.
- Cashier reporting is still mostly cards and recent sessions.
- The next useful layer is per-cashier/per-terminal/per-shift settlement history with variance reason, approval, proof, and reconciliation status.

Relevant evidence:

- `services/pos/drawer-dashboard.service.ts`
- `components/pos/CashDrawerManagementDashboard.tsx`
- `actions/pos/drawer-dashboard.actions.ts`
- `prisma/schema.prisma`

Payroll And HRIS-Adjacent Functionality

Confirmed:

- Payroll has strong proof-oriented surfaces: runs, register tie-out, payments, declarations, payslips, employee balances, payment reconciliation, and close evidence.
- Prisma includes `PayrollEmployee`, `PayrollContract`, attendance snapshots, periods, runs, run lines, payslips, declarations, payment batches, allocations, and employee balance cases.
- Employee identity, contracts, compensation controls, attendance snapshots, payment destination changes, and certified payroll inputs already exist under payroll namespaces.

Important gaps:

- There is no standalone HRIS bounded context.
- People-data routes and services are currently payroll-owned.
- Org/manager scope is limited and does not represent a full effective-dated org hierarchy.
- There is no first-class leave policy, leave balance, leave request, position history, employee document center, or manager self-service module.
- Payroll pages often use fixed limits without a reusable history/pagination pattern.

Relevant evidence:

- `services/payroll/*`
- `components/payroll/*`
- `app/[locale]/(dashboard)/dashboard/payroll/*`
- `docs/HR-Payroll/*`
- `what-next/payroll/*`
- `prisma/schema.prisma`

Payment And Reconciliation

Confirmed:

- Payment and reconciliation have a mature data foundation: `Payment`, `PaymentTransaction`, provider accounts, provider events, statement records, match records, suspense, exceptions, reconciliation runs, evidence, and certification flows.

- Finance payments show recent payment activity.
- Durable reconciliation services support matching, signing, suspense, proof trails, and certificates.

Important gaps:

- Some payment surfaces are recent/aggregate views, not complete payment transaction histories.
- There are two levels of truth: a capture-readiness model and a durable reconciliation evidence kernel. The UI should clearly label which source controls close decisions.
- Payment proof should become a transaction timeline linking internal payment, provider event, statement line, match decision, suspense/exception, ledger posting, reconciliation run, and certificate.

Relevant evidence:

- `services/payments/payment-reconciliation-workbench.service.ts`
- `services/reconciliation/payment-reconciliation-dashboard.service.ts`
- `components/finance/PaymentReconciliationWorkbench.tsx`
- `components/finance/FinanceSpecializedLedgerSurfaces.tsx`
- `prisma/schema.prisma`

Purchasing And AP Controls

Confirmed:

- AP control is one of the strongest operational foundations.
- Supplier invoices, duplicate fingerprints, three-way matches, supplier payment approvals, bank-detail changes, released payments, ledger blockers, reconciliation identifiers, evidence hashes, source links, and posting batches are already present.
- Purchase-order management includes search, status filtering, status history, receipt lifecycle, and export.

Important gaps:

- AP workbench lists are bounded and not yet complete transaction histories.
- A full AP control history should expose lifecycle stages from PO to receipt to invoice to match to payment to reconciliation to posting.

Relevant evidence:

- `services/purchasing/ap-control.service.ts`
- `components/purchasing/APControlWorkbench.tsx`
- `components/ui/groups/purchase-orders/PurchaseOrderManagement.tsx`
- `prisma/schema.prisma`

Sales And Receivables

Confirmed:

- Per-customer order history exists.
- Finance receivables show aging, top customers, and recent inbound payments.
- Daily sales reporting derives aggregate sales/payment information.
- Customer ledger entries exist in Prisma.

Important gaps:

- There is no confirmed organization-wide sales/receivables transaction-history table equivalent to a full AR control queue.
- Receivables currently lean on balances and recent payments, not complete open-item and allocation history.
- A polished AR statement should not be built until receipt allocation, credits, refunds, bad debt, write-offs, due dates, and GL tie-outs are explicit.

Relevant evidence:

- `services/finance/finance-dashboard.service.ts`
- `components/finance/FinanceSpecializedLedgerSurfaces.tsx`
- `services/analytics/financial-analytics.service.ts`
- `prisma/schema.prisma`

Close And Audit Evidence

Confirmed:

- Close assurance persists runs, checklist items, findings, evidence, comments, reviews, evidence graphs, and close packs.
- Accountant portal surfaces blockers, source links, combined audit/control events, and trust-pack export.
- `AccountingSourceLink`, `LedgerPostingBatch`, `JournalEntry`, `BusinessEvent`, and close models provide a natural evidence spine.

Important gaps:

- Histories are bounded rather than paginated.
- Proof trails currently cover only selected subject types.
- Some domains need explicit evidence subject contracts before proof badges can be honestly promised everywhere.

Relevant evidence:

- `services/accounting/close-assurance.service.ts`
- `services/accounting/data-trust.service.ts`

- components/accounting/CloseAssuranceCenter.tsx
- components/accounting/AccountantPortal.tsx
- services/evidence/evidence-contracts.ts
- prisma/schema.prisma

Recommended Shared Pattern: TransactionHistoryWorkbench

Recommendation

Create a shared, server-backed TransactionHistoryWorkbench for movement-style operational histories. It should standardize page structure, filtering behavior, pagination, row selection, export semantics, and proof-drawer behavior, while leaving columns, permissions, data ownership, and business logic inside each domain.

Suggested file shape:

```
components/dashboard/history/
  TransactionHistoryWorkbench.tsx
  TransactionHistoryTable.tsx
  TransactionDetailDrawer.tsx
  TransactionFilterBar.tsx
  transaction-history.types.ts
```

Domain modules should provide adapters such as:

- inventoryMovementHistoryColumns
- apHistoryFilters
- cashierSettlementDrawerContent
- payrollBatchHistoryAdapter
- supplierPayableHistoryAdapter
- customerReceivableHistoryAdapter

Do not build a universal write ledger. The shared layer should be a read-model and UX contract, not a new source of truth.

Canonical Page Anatomy

Every history surface should follow the authenticated dashboard command-center style:

1. Command brief: current state, scope, organization/location/period, "as of" time, primary action, proof access.
2. KPI/status strip: three to five stable metrics using the same filters as the table.
3. Action queue: only when intervention is needed.
4. Filter bar: search, date range, primary entity filter, status/control filter, more filters, chips, reset, export.
5. Workbench table: the main operational surface.
6. Detail/proof drawer: source, evidence, lifecycle, actors, hashes, postings, reconciliation, corrections.
7. Secondary analytics: trends, aging, mix, or volume after the operational table.

Every screen should answer:

- What is the state?
- What is the risk?
- What is the action?
- What is the proof?

Shared Row Contract

Every history table should expose these roles where applicable:

Column role	Requirement
Effective time	Business event timestamp in organization timezone; recorded time stays in drawer
Event/reference	Human reference plus event type; never raw ID alone
Subject	Item, supplier, customer, cashier, employee group, provider, or account
Context	Location, terminal, payroll run, purchase order, sales order, drawer, provider account
Effect	Signed quantity or amount with unit/currency
Business state	Draft, posted, paid, completed, returned, cancelled
Control state	Matched, unreconciled, blocked, stale, certified
Proof	Compact evidence grade/freshness badge
Actions	Inspect, view source, resolve, correct, reverse, approve when authorized

Data Contract

Every read model should return:

```

type TransactionHistoryResult<Row, Summary> = {
  rows: Row[]
  nextCursor: string | null
  appliedFilters: Record<string, unknown>
  summary: Summary
  asOf: string
  partialSources: Array<{
    source: string
    reason: string
    severity: "info" | "warning" | "blocked"
  }>
}

```

Rules:

- Use server filtering, sorting, and cursor pagination ordered by `(effectiveAt DESC, id DESC)`.
- Default to 50 rows; offer 25, 50, and 100.
- Preserve filters and selected row in URL query parameters.
- Export the full server-filtered result, not only loaded browser rows.
- Do not use UI-derived arithmetic as business truth.
- Do not enable bulk selection unless a real bulk action exists.
- Separate direction from risk. A normal outbound sale is not a danger state; theft, expiry, failed posting, blocking variance, or unresolved exception can be risk states.

Proposal Matrix

Proposal	Build decision	Primary business use case	Main users	Source of truth	UI pattern	Value	Key risks	Implementation path	Test
Supplier detail transaction history	Build as supplier activity drawer/page	Explain supplier activity across PO, receipt, invoice, return, credit, and payment	AP, procurement, controller, owner, auditor	Supplier service plus PO, receipt, invoice, payment, ledger, source-link services	Supplier detail history tab plus drawer	Faster supplier dispute handling and better AP visibility	Duplicate AP truth, stale balances, bank-data exposure	Start with read-only paginated supplier activity adapter	Tenants: pagination, balance redaction, compliance
Supplier payable transaction history	Build after AP contract	Statement with opening balance, invoices, payments, allocations, aging, closing balance	AP, treasury, controller, auditor	SupplierInvoice, SupplierPayment, SupplierPaymentAllocation, SupplierLedgerEntry, AccountingSourceLink	Payable statement table with proof drawer	Cash planning, duplicate-payment detection, AP-to-GL reconciliation	Running balance errors, overclaiming accounting maturity	Add payable read model; tie opening + movement = closing	AP controls: allocation, due-date reversals
Client/customer detail transaction history	Build as customer activity history	Explain sales, receipts, returns, refunds, credits, and adjustments for one customer	AR, sales ops, support, controller	Customer service plus sales, payment, returns, ledger, source-link services	Customer detail history tab plus drawer	Better support, credit review, dispute resolution	Privacy and credit exposure, incomplete AR semantics	Replace recent analytics lists with paginated read model	Customer search filters, drilldown
Client/customer receivables history	Build after AR foundations	Open item and customer statement history with due date, allocation, aging	AR, collections, owner, controller	AR service over customer ledger, invoices/orders, payments, allocations, GL links	Receivable statement and control queue	DSO reduction, collection prioritization, dispute clarity	Treating orders as invoices, double counting receipts	Create receipt allocation, credit/write-off, and due-date contracts first	Open aging credit approval reconciliation
Cashier daily transaction history	Build by extending cash drawer dashboard	Daily cashier/terminal/session operational statement	Cashier, supervisor, branch manager, controller	POS session, sales, payments, cash drawer transactions, drawer service	Sessions tab plus transaction journal and drawer	Shift accountability, faster cash close, variance explanation	Shared drawer attribution, punitive misuse, offline ambiguity	Add cashier history route/read model with cursor pagination	Session: own-view access, math, tests
Cashier movement and settlement history	Build as high-priority P1	Opening float, cash sales, refunds, payouts, cash-in/out, counted cash, variance, approval	Supervisor, controller, owner, internal audit	POS/cash drawer services and reconciliation services	Settlement packet with proof drawer	Strong physical cash control and daily close evidence	Electronic payments mixed with cash, missing approvals	Add settlement state machine and supervisor sign-off	Expected counts: reconciliation approval threshold, session
Payroll movement history	Build as controlled lenses	Trace payroll period/run/payment/declaration/proof movements	Payroll admin, HR, treasury, controller, employee self-service for own records	Payroll services, certified snapshots, payment/declaration/accounting proof	Three lenses: run lifecycle, payment proof, employee balance/history	Dispute resolution, payroll close assurance, audit readiness	High privacy risk, salary leakage, oversized exports	Start with aggregate run/batch history; gate employee drilldown	Fresh redaction, manual export, correctness

Proposal	Build decision	Primary business use case	Main users	Source of truth	UI pattern	Value	Key risks	Implementation path	Test
Inventory movement improvements	Improve existing surface	Complete stock movement and valuation traceability	Warehouse, inventory manager, controller, owner	Inventory transaction/read services	Existing movement page migrated to shared workbench	Better stock accountability and less false completeness	KPI/table mismatch, capped rows, enum drift	Fix filter-summary consistency, cursor pagination, proof drawer, write-off support	No-duplicate cursor enum KPI-filter export compliance
Payment and reconciliation movement history	Build as P0/P1 proof timeline	Explain payment from internal record to provider event, match, suspense, posting, certificate	Treasury, reconciliation analyst, support, controller	Payment/reconciliation services and source links	Transactions, runs, suspense tabs with proof drawer	Reduces unidentified cash and close delays	Confusing capture model with durable certified truth	Unify read model labels and proof timeline	Match manual: appropriate suspense block reduction
Stock adjustment and write-off history	Build as exception register	Explain counts, damage, expiry, theft, shrinkage, corrections, write-offs	Warehouse manager, controller, owner, auditor	Inventory adjustment/count services, inventory transactions, ledger/source links	Exception queue plus detail/proof drawer	Protects valuation, COGS, margin, physical accountability	Sensitive allegations, incorrect costs, unsupported retro edits	Add adjustment/write-off route and include <code>WRITE_OFF</code> in filters	Make reason required period close
Purchasing/AP control history	Extend existing AP workbench	Lifecycle from PO to receipt to invoice to match to payment to posting	Buyer, receiver, AP, approver, treasury, controller	AP control service, PO, receipt, invoice, payment, posting, reconciliation	AP history tabs: invoices, disbursements, blockers	Prevents duplicate invoices, bank-fraud, overpayment, cut-off errors	Giant unreadable table, country-pack simplification	Add server filters, pagination, and lifecycle drawer to AP workbench	Segregate duties tolerate change payment posting
Sales/receivables control history	Build after AR foundation	Portfolio queue for billing, collection, dispute, credit, write-off, allocation	AR, sales ops, controller, owner	Sales/AR service, customer ledger, payment allocations, GL links	Receivables control queue and drawer	Exposes revenue-to-cash leakage and overdue risk	Misstating revenue or receivables maturity	Define AR source service and open-item contract	Order semantic dates, refund notes,
Close and evidence movement history	Enhance existing close assurance	Period evidence delta, readiness changes, findings, waivers, review trail	Controller, accountant, CFO, auditor	Close assurance, data trust, evidence services	Close-run timeline and evidence-delta table	Shortens future close and improves audit prep	Users may mistake system assurance for statutory certification	Add paginated evidence history and proof subject coverage	Waive segregation evidence retention verification review

Prioritized Roadmap

P0: Foundation And Data Trust

1. Define `TransactionHistoryWorkbench` and the shared server result contract.
2. Add server/manual mode to table handling with cursor pagination and full-query export.
3. Fix inventory movement filter-summary consistency and add `WRITE_OFF` coverage.
4. Define proof subject contracts for inventory transactions, POS sessions, AP invoices, sales/receivables, and payroll batches before showing proof badges.
5. Create or clarify AR service boundaries for receipt allocation, credit notes, refunds, bad-debt/write-off, due dates, and GL tie-outs.
6. Publish an internal hashing/security usage inventory.

P1: Highest Operational Value

1. Payment reconciliation transaction proof timeline.
2. Cashier movement and settlement history.
3. Purchasing/AP lifecycle history.
4. Inventory adjustment and write-off exception register.
5. Supplier payable statement and customer receivable statement once service contracts are ready.

P2: Control Depth And Product Polish

1. Sales/receivables portfolio control queue.
2. Payroll movement lenses with strict redaction and fresh-auth controls.
3. Close-run and evidence-delta history.
4. Proof drawer normalization across supported subject types.
5. Mobile, keyboard, EN/FR, timezone, partial-source, and export consistency hardening.

P3: HRIS Roadmap

1. Extract `services/hris/`, `actions/hris/`, `/dashboard/hr/*`, and `hris.*` permissions while wrapping existing payroll-owned storage.
2. Add effective-dated employment assignment: org unit, position, location, manager, job, cost center, reason.
3. Add HR lifecycle events and employee 360 timeline.
4. Add time/leave policy, request, balance, approval, and correction workflow.

5. Add employee self-service and manager-scoped workflows after privacy/RBAC gates pass.

Security And Hashing Analysis

What Hashing Is Useful For

Hashing is useful when the system needs a stable fingerprint of content or a lookup key for high-entropy secrets:

- Content fingerprints for snapshots, reports, evidence bundles, country packs, business events, and fiscal documents.
- Idempotency checks when the input has enough entropy and the digest is recomputed at the trust boundary.
- High-entropy token registry lookups, such as random receipt token IDs or random invite tokens.
- Drift detection when the expected digest comes from a trusted source.

What Hashing Does Not Protect

Hashing does not:

- Encrypt data.
- Hide low-entropy values such as phone numbers, six-digit OTPs, small salary ranges, or predictable identifiers.
- Prove who produced a record.
- Prevent replay of a leaked bearer token.
- Prevent mutation if both the record and hash are stored in the same mutable database.
- Replace RBAC, redaction, retention, signatures, or audit controls.

Password Hashing

Confirmed:

- Passwords use Argon2id through `lib/password.ts` and seeding also uses Argon2id.

Recommendation:

- Keep Argon2id for passwords.
- Add `needsRehash` style upgrade logic on successful login if not already present.
- Do not replace password hashing with SHA-256, SHA-512, or HMAC.
- Keep breached-password screening separate from password storage. HIBP SHA-1 prefix usage is a protocol-specific lookup, not password storage.

Integrity Hashes And Checksums

Confirmed:

- Stoquify uses SHA-256 for many content fingerprints, source hashes, snapshot hashes, country-pack hashes, business event payload hashes, and report/filter fingerprints.

Recommendation:

- SHA-256 is appropriate for content fingerprints and checksums.
- Always describe these as "fingerprints" or "checksums," not as proof of authenticity by themselves.
- Recompute hashes at the service boundary rather than trusting a caller-provided digest.
- Validate evidence hashes with a strict format such as `sha256:<64 lowercase hex chars>` when they are intended to be verifiable digests.

HMAC Signing And Keyed Hashes

Confirmed:

- Public receipt tokens and some abuse-prevention flows use HMAC-SHA-256 or hashed high-entropy token registry patterns.
- Payment webhooks include HMAC-style verification in relevant adapter code.

Recommendation:

- Keep HMAC-SHA-256 for webhook authenticity, public-token signing, and keyed pseudonymous lookup.
- Use purpose-scoped secrets and include a key ID where rotation is expected.
- Use HMAC, not bare SHA-256, for low-entropy identifiers that need lookup, such as phone numbers, OTPs, salary bands, or predictable external references.
- Do not place shared HMAC secrets in browser/offline POS code.

Token Security

Confirmed concern:

- Some recovery/verification token helpers use `Math.random()` and plaintext token storage patterns.

Recommendation:

- Use cryptographic randomness: `randomBytes` for reset links and `randomInt` for numeric OTPs.
- Store only a keyed digest of OTPs/reset tokens where the original value does not need to be recovered.
- Apply short expiry, rate limits, purpose scoping, user scoping, and revocation.
- Keep invite tokens high entropy, but store only a digest where possible.

Offline POS Evidence

Confirmed concern:

- Offline POS hash chains can detect local sequencing changes only if the signer is trusted. A browser-generated SHA-256 chain is not device authenticity.

Recommendation:

- For fiscal or non-repudiation claims, use device-specific public/private key signing such as Ed25519.
- Register device public keys server-side.
- Sign organization, device, terminal, sequence, event type, payload hash, previous hash, and capture timestamp.
- Verify the signature before accepting offline envelopes.
- Treat existing offline hashes as checksums unless signature verification is implemented.

Audit Log Integrity

Confirmed concern:

- Generic audit logs are database records and should not be treated as immutable just because they exist.

Recommendation:

- Restrict application DB role from updating/deleting/truncating audit logs.
- Add append-only or WORM-style external anchoring for critical logs.
- For critical records, use a chained MAC over canonical audit data and externally anchored checkpoints.
- Remember that HMAC stored inside the same compromised application boundary is not non-repudiation.

Public Receipt Links

Confirmed:

- Public receipt token design is generally sound when tokens are high entropy, signed, time-bound, organization-bound, and checked against a token registry.

Recommendation:

- Add `Cache-Control: private, no-store` and `Referrer-Policy: no-referrer` headers on public receipt responses.
- Redact receipt `token` query parameters from edge and APM logs.
- Add key IDs and active/previous keys if token rotation must preserve outstanding links.
- Continue redacting public receipt contact fields.

Practical Security Priority

1. Replace `Math.random()` security tokens with cryptographic randomness and digest storage.
2. Move low-entropy bare SHA-256 identifier hashes to HMAC-SHA-256 with purpose-scoped rotating keys.
3. Clarify evidence hash language and strict digest validation.
4. Implement server-side offline POS signature verification before making strong evidence claims.
5. Harden audit-log immutability with DB permissions and external checkpoints.
6. Add key IDs, rotation policy, and secret preflight coverage for auth, receipt, identity-abuse, token, and encryption secrets.

HRIS Proposal

Honest Current-State Assessment

HRIS functionality is partially present, but not as a standalone HRIS module. The current system has a payroll-adjacent HRIS foundation:

- Employee identity and lifecycle fields.
- Contracts.
- Compensation controls.
- Payment destination changes.
- Attendance snapshots.
- Payroll readiness gates.
- Certified payroll run inputs.
- Payroll payment, declaration, payslip, and accounting proof.

However, these capabilities live under payroll models, payroll services, payroll routes, and payroll permissions. There is no confirmed standalone `services/hris/`, `actions/hris/`, `/dashboard/hr/*`, or `hris.*` permission family.

What Appears Missing

The missing HRIS capabilities are:

- Standalone HRIS bounded context.
- Effective-dated org assignment.
- Position and reporting-line history.
- First-class leave policy, balances, requests, approvals, and corrections.
- Employee document center with retention and secure access.
- Employee profile self-service.
- Manager team workflows.

- HR-specific RBAC separated from payroll permissions.
- HR lifecycle timeline independent of payroll runs.

Recommended HRIS Direction

Build HRIS as a phased bounded context, not a big-bang rewrite and not a permanent payroll-only extension.

Positioning:

HRIS owns people and employment truth. Payroll consumes certified snapshots. Accounting owns money truth. Assurance proves the chain.

Minimum Viable HRIS Slice

The smallest useful HRIS slice should include:

- Tenant-scoped employee profile and lifecycle status.
- Effective-dated employment assignment with org unit, location, position, manager, cost center, and movement reason.
- Contract activation, amendment, and termination with evidence.
- Effective-dated compensation with maker-checker approval.
- Append-only HR lifecycle/movement events.
- Employee list, employee detail, and unified employee history.
- Certified HRIS-to-payroll snapshot.
- Field-level redaction for salary, tax/social identifiers, payment destinations, documents, and sensitive leave reasons.

Do not include recruitment, performance reviews, learning management, broad benefits administration, or workforce AI in the first slice.

HRIS Implementation Path

1. Boundary extraction:
 - Introduce HRIS service/action/route/permission namespaces.
 - Initially wrap existing payroll-owned data to avoid dual-write risk.
2. Employment assignment:
 - Add effective-dated assignment model and movement reasons.
 - Replace location-only manager assumptions with effective-dated reporting lines.
3. HR lifecycle history:
 - Create employee 360 timeline and movement register.
 - Show effective date separately from recorded/approved date.
4. Time and leave:
 - Add schedules, work calendars, leave policy, balance, request, approval, overtime, absence, and correction.
 - Convert approved results into immutable payroll attendance snapshots.
5. Self-service:
 - Add employee profile/document/leave requests and manager team workflows.
6. Cutover:
 - Backfill HRIS ownership, certify tenant diffs, and restrict payroll from reading mutable HR tables directly.

HRIS Tests Before Release

- Tenant isolation.
- HR vs payroll permission separation.
- Manager scope negative tests.
- Salary and payment destination redaction.
- Document access and retention checks.
- Effective-dated assignment correctness.
- Payroll snapshot immutability.
- Retroactive change handling.
- Employee self-service own-record access only.
- Browser, mobile, accessibility, EN/FR, and export privacy tests.

Implementation Detail: First Three Build Slices

Slice 1: Workbench Foundation And Inventory Pilot

Goal:

- Prove the shared `TransactionHistoryWorkbench` with one real existing surface.

Scope:

- Add shared history types, filter schema, table shell, detail drawer shell, and server pagination contract.

- Migrate inventory movements to the new workbench.
- Fix summary/table filter parity.
- Add `WRITE_OFF` and enum alignment.
- Add no-active-organization and query-error states.

Release tests:

- Existing inventory movement behavior still works.
- Type/date/item/location filters affect table and KPIs consistently.
- Cursor pagination has no duplicate or missing rows.
- Export includes all server-filtered rows.
- Empty, error, loading, partial-source, and no-organization states render properly.

Slice 2: Cash And Payment Proof

Goal:

- Deliver the highest daily operational value: cash discrepancies and payment reconciliation proof.

Scope:

- Add cashier/session settlement history.
- Add payment transaction proof timeline.
- Label capture-readiness vs durable reconciliation truth.
- Add proof drawers for payment transaction, provider event, match, suspense, posting, and certificate.

Release tests:

- Cash expected vs counted reconciliation.
- Variance reason and approval thresholds.
- Cashier own-session vs manager access.
- Payment match/suspense state transitions.
- Manual match segregation of duties.
- Provider PII redaction.
- Close blocker behavior for unresolved suspense.

Slice 3: AP/AR Control Histories

Goal:

- Convert working-capital risk into operational tables.

Scope:

- AP lifecycle history from PO to payment to reconciliation to posting.
- Supplier payable statement.
- AR service foundation for receipt allocation, credits, refunds, bad debt/write-offs, due dates, and GL source links.
- Customer receivable statement after AR service contract is complete.

Release tests:

- Supplier statement opening + movements = closing.
- Supplier subledger ties to AP control account.
- Duplicate invoice fingerprint behavior.
- Bank-detail change hold.
- Customer open-item tie-out.
- Receipt allocation and reversal behavior.
- Credit/write-off approval and evidence.
- Export completeness and redaction.

Design Rules For New History Surfaces

Use:

- Dark dashboard command-center style.
- Compact, scannable tables.
- Command brief.
- KPI/status strip.
- Action queue where intervention is required.
- Workbench table.
- Detail/proof drawer.
- Evidence strip when proof is central.
- Server pagination and full-query export.
- Service-owned summaries and rows from the same filter contract.

Avoid:

- Decorative dashboards.
- Duplicated cards.
- Route-local palettes.
- UI-derived business truth.
- Client-only filtering for large ledgers.
- Capped "recent" lists presented as complete histories.
- Static demo transaction cards.
- Editable history rows.
- Universal polymorphic write ledger.
- Raw IDs and hashes as the primary user-facing evidence language.

Risks And Tradeoffs

Where It May Not Be Worth Building Yet

- Full customer receivable statements should wait until AR allocation, credit, refund, write-off, and GL tie-out semantics are mature.
- Standalone HRIS should not jump ahead of cash, payment, AP, AR, and inventory control improvements unless payroll/HR is the immediate commercial priority.
- A universal movement table should not be built if it forces every domain into generic columns and loses domain meaning.
- Proof badges should not be shown for domains not covered by evidence contracts and permission mappings.
- Payroll employee-level history should not be broadly exported; privacy and redaction risks are too high.

Main Tradeoffs

- Shared UI pattern vs domain specificity: share shell and behavior, not business truth.
- Fast visible pages vs reliable ledgers: prioritize service-owned read models and pagination before UI expansion.
- More proof vs more privacy risk: use drawers, permission gates, redaction, and audit logging.
- Broad HRIS ambition vs product focus: phase HRIS as payroll-grade people truth before broad HR suite features.

Verification Checklist Before Any Release

Functional:

- Server filters, summary, table, drawer, and export use the same normalized filters.
- Cursor pagination has stable ordering and no duplicate rows.
- Back/refresh preserves filters and selected row.
- Empty, partial, error, no-organization, and loading states are explicit.
- Exports are generated server-side and include the complete filtered result.

Accounting/control:

- Opening + movements = closing where a balance is shown.
- Subledger ties to control account where accounting claims are made.
- Posted records are corrected/reversed, not edited.
- Source links resolve to authoritative records.
- Close blockers are generated for unresolved material exceptions.

Security/RBAC:

- Tenant isolation tests cover all read models.
- Permission tests cover table, drawer, export, and action buttons.
- Redaction tests cover salary, payment destination, provider references, bank data, customer credit, supplier bank changes, tax/social identifiers, and sensitive leave reasons.
- Hashes and tokens use the correct primitive for the context.
- Security tokens use cryptographic randomness and expiry.
- Low-entropy lookups use HMAC, not bare SHA-256.

UX/accessibility:

- Works at 320px and common desktop widths.
- Dark dashboard tokens are used consistently.
- Keyboard navigation and focus restoration work for drawers.
- Numeric values align and use consistent currency/unit formatting.
- Dates respect organization timezone and distinguish effective, recorded, due, settlement, and pay dates.
- EN/FR labels and statuses fit without overlap.

Final Recommendation

Proceed, but do it as a product architecture improvement rather than a set of copied pages.

The best first move is:

1. Define the shared `TransactionHistoryWorkbench` contract.
2. Use inventory movements as the pilot and fix its known completeness issues.
3. Build cash settlement and payment reconciliation proof next.
4. Extend AP and AR histories after service-owned source-of-truth contracts are clear.
5. Treat HRIS as a phased bounded-context roadmap, not as a claim that the current payroll module is already a complete HRIS.

This will make Stoquify feel more operationally trustworthy without overstating the current system or weakening the service-owned truth model.